

HTML SVG

A Practical, From-Zero Learning Guide

Part 5 — SVG Text & Typography

Text is a real SVG element, not merely HTML text placed over a picture. You can position, style, rotate, align and even place SVG text along a path.

Goal: Learn how SVG text works and build labels, logos, badges, UI graphics and curved text.

1. The Element

The basic SVG text element is `<text>`.

```
<svg viewBox="0 0 400 150">
  <text x="40" y="90">Hello SVG</text>
</svg>
```

The **x** and **y** attributes position the text. Unlike a rectangle, the y coordinate normally corresponds to the text's baseline.

2. Understanding the Baseline

SVG text is positioned using typographic concepts. The baseline is the imaginary line on which most letters sit.

```
<text x="30" y="100"
      font-size="40">
  Hello
</text>
```

The visible letters extend above and sometimes below the baseline. This is why **y=100** does not mean the top of the text is at y=100.

3. Font Size

```
<text x="30" y="70" font-size="20">Small</text>
<text x="30" y="120" font-size="40">Large</text>
```

You can use CSS units or numeric values. SVG commonly uses user units when no unit is supplied.

4. Font Family

```
<text x="30" y="90"
      font-family="Arial, sans-serif"
      font-size="36">
  SVG Text
</text>
```

Use a font stack so the browser can fall back to another font if the preferred font is unavailable.

5. Font Weight

```
<text x="30" y="90"
      font-size="40"
      font-weight="700">
  Bold SVG
</text>
```

You can use values such as **normal**, **bold**, or numeric weights when the font supports them.

6. Font Style

```
<text x="30" y="90"
      font-size="40"
      font-style="italic">
  Italic
</text>
```

CSS font properties work with inline SVG.

7. Fill and Stroke on Text

```
<text x="40" y="100"
```

```

    font-size="55"
    fill="gold"
    stroke="black"
    stroke-width="2">
STAR
</text>

```

Text is another SVG graphical element, so it can use fill and stroke.

8. text-anchor

text-anchor controls horizontal alignment relative to the x coordinate.

```

<text x="200" y="70"
    text-anchor="start"
    font-size="25">Start</text>

<text x="200" y="110"
    text-anchor="middle"
    font-size="25">Middle</text>

<text x="200" y="150"
    text-anchor="end"
    font-size="25">End</text>

```

The main values are **start**, **middle** and **end**.

9. dominant-baseline

This property controls how text is aligned vertically relative to its position.

```

<text x="30" y="100"
    dominant-baseline="middle"
    font-size="35">
    Centered
</text>

```

It is useful when placing labels inside buttons, badges and shapes.

10. Centering Text in a Box

```

<rect x="50" y="40"
    width="300" height="100"
    rx="15" fill="#7c3aed"/>

<text x="200" y="90"
    text-anchor="middle"
    dominant-baseline="middle"
    fill="white"
    font-size="28">
    Button
</text>

```

This is a common pattern for SVG UI graphics.

11. Letter Spacing

```

<text x="30" y="90"
    font-size="35"
    letter-spacing="5">
    SPACED
</text>

```

Use **letter-spacing** to control the space between characters.

12. Word Spacing

```
<text x="30" y="90"
      font-size="30"
      word-spacing="12">
  SVG makes graphics
</text>
```

Use **word-spacing** when you want larger gaps between words.

13. textLength

The **textLength** attribute lets you request a particular rendered length.

```
<text x="30" y="90"
      font-size="35"
      textLength="300">
  Stretched Text
</text>
```

The browser adjusts spacing or glyph scaling according to the text-length settings.

14. lengthAdjust

```
<text x="30" y="90"
      font-size="35"
      textLength="300"
      lengthAdjust="spacingAndGlyphs">
  Flexible Text
</text>
```

Common values include **spacing** and **spacingAndGlyphs**.

15. The Element

<tspan> allows parts of one text element to have different styling or positioning.

```
<text x="30" y="90" font-size="36">
  Hello
  <tspan fill="tomato" font-weight="bold">
    SVG
  </tspan>
</text>
```

This is useful for highlighted words and multi-style labels.

16. Positioning a tspan

```
<text x="30" y="80" font-size="32">
  First
  <tspan x="30" dy="40">Second</tspan>
  <tspan x="30" dy="40">Third</tspan>
</text>
```

dy changes the vertical position relative to the current text position. You can also use **dx** for horizontal offsets.

17. Absolute tspan Positioning

```
<text font-size="30">
  <tspan x="30" y="60">One</tspan>
  <tspan x="30" y="110">Two</tspan>
  <tspan x="30" y="160">Three</tspan>
</text>
```

A **tspan** can specify its own x and y positions.

18. Multi-line SVG Text

```

<text x="30" y="60" font-size="24">
  <tspan x="30" dy="0">First line</tspan>
  <tspan x="30" dy="35">Second line</tspan>
  <tspan x="30" dy="35">Third line</tspan>
</text>

```

SVG text does not automatically wrap like a normal HTML paragraph, so tspan's are often used for controlled multi-line text.

19. CSS Classes for Text

```

<style>
.title {
  font-size: 42px;
  font-weight: 700;
  fill: #111827;
}
.subtitle {
  font-size: 18px;
  fill: #64748b;
}
</style>

<text class="title" x="30" y="70">CraveCart</text>
<text class="subtitle" x="30" y="105">Food delivered simply</text>

```

Using classes is cleaner than repeating long style attributes.

20. Transforming Text

```

<text x="30" y="80"
  font-size="35"
  transform="rotate(-10 30 80)">
  Tilted
</text>

```

SVG text can use the same transformations you learned in Part 3.

21. Rotating Individual Letters

```

<text x="30" y="100" font-size="40">
  H
  <tspan transform="rotate(15 65 100)">i</tspan>
  !
</text>

```

For precise per-character effects, tspan's give you more control.

22. textPath — Text Along a Path

One of the most powerful SVG text features is placing text along an existing path.

```

<svg viewBox="0 0 500 200">

  <defs>
    <path id="curve"
      d="M 50 150 Q 250 10 450 150"/>
  </defs>

  <text font-size="28">
    <textPath href="#curve"
      startOffset="50%"
      text-anchor="middle">
      Text follows a curve
    </textPath>
  </text>

```

```
</svg>
```

The text follows the geometry of the referenced path.

23. startOffset

startOffset controls where the text begins along the path.

```
<textPath href="#curve"
          startOffset="25%">
  Quarter Position
</textPath>
```

It can be expressed as a length or percentage.

24. Curved Logo Example

```
<svg viewBox="0 0 500 300">
  <defs>
    <path id="logoArc"
          d="M 80 170
            A 170 170 0 0 1 420 170"/>
  </defs>

  <text font-size="30"
        font-weight="700"
        letter-spacing="3">
    <textPath href="#logoArc"
              startOffset="50%"
              text-anchor="middle">
      CRAVE CART
    </textPath>
  </text>
</svg>
```

This technique is common in badges, seals, logos and circular labels.

25. Text Along a Circle

```
<svg viewBox="0 0 300 300">
  <defs>
    <path id="circlePath"
          d="M150 150
            m-100 0
            a100 100 0 1 1 200 0
            a100 100 0 1 1 -200 0"/>
  </defs>

  <text font-size="18"
        text-anchor="middle">
    <textPath href="#circlePath"
              startOffset="50%">
      SVG CIRCULAR TEXT
    </textPath>
  </text>
</svg>
```

A circular path can be used as the text baseline.

26. text-decoration

```
<text x="30" y="90"
      font-size="32"
      text-decoration="underline">
  Underlined
</text>
```

SVG text can use CSS text-decoration properties supported by the browser.

27. Writing Modes

SVG can support different writing directions and vertical layouts.

```
<text x="80" y="40"
      writing-mode="vertical-rl"
      font-size="28">
  VERTICAL
</text>
```

Writing-mode is particularly useful for multilingual interfaces and vertical typography.

28. Direction and Text Alignment

```
<text x="250" y="90"
      direction="rtl"
      text-anchor="middle"
      font-size="30">
  ■■■■
</text>
```

SVG supports right-to-left text and other international text-layout requirements.

29. Text Selection and Accessibility

Inline SVG text is real document text, unlike text that has been converted into an image. This can be useful for accessibility and searchability.

```
<svg role="img"
      aria-labelledby="title desc"
      viewBox="0 0 300 120">
  <title id="title">Sales chart</title>
  <desc id="desc">A simple chart showing increasing sales.</desc>

  <text x="20" y="70">Sales increased</text>
</svg>
```

For meaningful graphics, **<title>** and **<desc>** can provide accessible descriptions.

30. HTML Text vs SVG Text

Feature	HTML text	SVG text
Normal paragraphs	Excellent	Not ideal
Responsive document text	Excellent	More specialized
Graphics labels	Possible	Excellent
Text along path	No native equivalent	Excellent
Per-shape transformations	Limited	Excellent
Complex page layout	Excellent	Not the main purpose

31. Mini Project — SVG Badge

```
<svg viewBox="0 0 400 400">
  <defs>
    <path id="badgePath"
          d="M200 200
              m-140 0
              a140 140 0 1 1 280 0
```

```

        a140 140 0 1 1 -280 0"/>
</defs>

<circle cx="200" cy="200" r="130"
        fill="#7c3aed"/>

<text fill="white"
        font-size="22"
        font-weight="700"
        letter-spacing="4">
  <textPath href="#badgePath"
            startOffset="50%"
            text-anchor="middle">
    LEARN SVG
  </textPath>
</text>

<text x="200" y="215"
        text-anchor="middle"
        dominant-baseline="middle"
        fill="white"
        font-size="40">
  SVG
</text>
</svg>

```

Try changing the path, text size, letter spacing and startOffset.

32. Mini Project — Card Title

```

<svg viewBox="0 0 500 180">
  <rect x="20" y="20"
        width="460" height="140"
        rx="20"
        fill="#f1f5f9"/>

  <text x="50" y="75"
        font-size="34"
        font-weight="700"
        fill="#111827">
    CraveCart
  </text>

  <text x="50" y="112"
        font-size="18"
        fill="#64748b">
    <tspan x="50" dy="0">Fresh food.</tspan>
    <tspan x="50" dy="27">Simple ordering.</tspan>
  </text>
</svg>

```

33. Common Mistakes

- Forgetting that y normally represents the text baseline.
- Expecting SVG text to wrap automatically like an HTML paragraph.
- Using text-anchor without considering the x coordinate.
- Using font families that are unavailable on the user's device.
- Forgetting that textPath needs a referenced path.
- Using SVG text for an entire webpage instead of using normal HTML.
- Ignoring accessibility for important graphical text.
- Converting text to paths when real text would be more useful.

34. Exercises

- Create a centered SVG heading inside a rectangle.
- Create a two-line label using `tspan`.
- Create a badge with text following a circular path.
- Create curved text above a logo.
- Create a vertical SVG label.
- Make one word a different color using `tspan`.
- Create an icon with an accessible title and description.
- Experiment with `text-anchor`: start, middle and end.
- Build a simple SVG restaurant logo using text and shapes.

35. Quick Reference

Element / Property	Purpose
<code><text></code>	Draw SVG text
<code><tspan></code>	Style or position part of text
<code><textPath></code>	Place text along a path
<code>x / y</code>	Position text
<code>dx / dy</code>	Offset text
<code>text-anchor</code>	Horizontal text alignment
<code>dominant-baseline</code>	Vertical alignment
<code>font-family</code>	Font selection
<code>font-size</code>	Text size
<code>font-weight</code>	Text weight
<code>font-style</code>	Italic/normal style
<code>letter-spacing</code>	Character spacing
<code>word-spacing</code>	Word spacing
<code>textLength</code>	Requested rendered length
<code>startOffset</code>	Starting position on <code>textPath</code>
<code>writing-mode</code>	Text writing direction/layout

36. What's Next?

Part 6 will cover **Clipping, Masking & Filters**: `<clipPath>`, `<mask>`, filter primitives, blur, shadows, color effects, compositing and practical image-effect projects.